AI Engineering and Claude Architect · Part II, Retrieval

RAG, Lesson 1

Why RAG Exists and When Not to Use It

A team spent three months building a retrieval pipeline for a 300-page handbook. A cached prompt holding the whole handbook would have taken an afternoon, might have cost less per question, and would never have failed to retrieve the right page.

Module	AI05 of 13
Lesson	1 of 7
Prerequisites	AI02 L6, AI03 L5–L6, AI04, HLD M21B (the RAG pipeline)
Exam relevance	RAG isn't a named domain. Embeddings are out of scope. The grounding and provenance ideas support TS 5.1 and TS 5.6
Reading time	~30 minutes

Contents

1. Why RAG Exists and When Not to Use It

Lesson 1 of 7 · RAG

Why RAG Exists and When Not to Use It

Key idea

RAG solves one specific problem: the model needs knowledge that won't fit in the prompt, or that changes, or that it was never trained on. Retrieval picks the relevant pieces at question time. But retrieval can fail, and every failure becomes a confident wrong answer. **So first ask whether you need retrieval at all: a small knowledge base can simply go in a cached prompt, and live data belongs behind a tool.**

> *Before: HLD M21B teaches the ingestion and query pipelines, RAG vs fine-tuning, and the five main failure modes. This lesson doesn't repeat them. Teaches: how to choose between stuffing, tools and RAG, the cost arithmetic, how real systems are shaped, and what RAG doesn't fix. Used by: all of AI05, AI06 L3, AI08 L5.*

The hook: the pipeline nobody needed

Your HR team wants an assistant over a 300-page handbook, updated monthly. The team plans the standard stack: chunking, embeddings, a vector database, hybrid search, a reranker and an evaluation harness.

Then someone asks how many tokens the handbook is. It's about 120,000.

Key idea

Anthropic's own guidance says that below about 200,000 tokens (about 500 pages), you can include the whole knowledge base in the prompt, with prompt caching. No chunking, no index, and no way to fail at retrieving the right page. A team should earn its way up to a pipeline, rather than starting there.

That threshold comes from Anthropic's Contextual Retrieval post. It's guidance, not a law, and current models have a 1M-token window. So measure quality on your own task.

Intuition: three ways to answer from a library

Approach	Library analogy	Good when
Stuff the prompt	Hand someone the whole book	The book is short
RAG	A librarian fetches the likely pages	The library is big
Tools	Telephone the department that knows	The answer changes by the minute

Most mature systems use more than one. A support agent might have its policy text cached, retrieve from a knowledge base, and call `lookup_order` for live data.

Choosing an approach

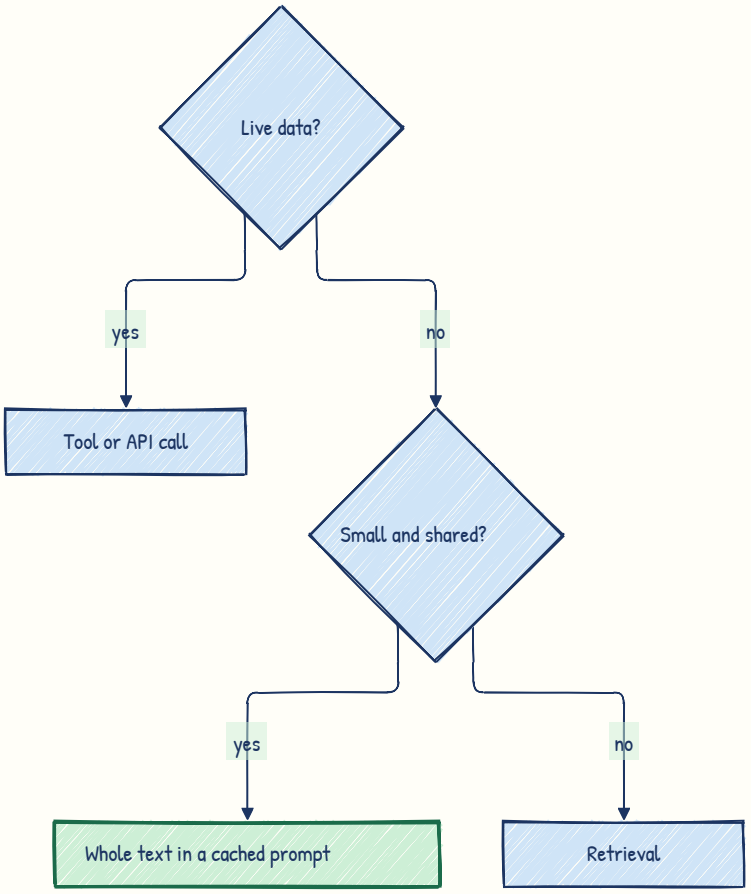


Figure 1. Choosing how to supply knowledge. Live data goes behind tools. Text under about 200,000 tokens that everyone may read goes in the prompt. Only large or permissioned collections need retrieval.

Factor	Stuff the prompt	RAG	Tool
Size	Small	Any	Any
Freshness	Next cache write	After re-indexing	Live
Per-user permissions	Hard, since all see the same text	Filter during search	Handled by the service
Failure mode	Attention dilution	**Missed retrieval**	Service errors
Build effort	Hours	Weeks	Days
Exact lookups	Model reads it	Weak unless hybrid	Excellent
Aggregations, such as "total sales"	No	No	**Yes, using SQL**

That last row catches many teams. RAG retrieves passages. It can't count or sum. "How many refunds last quarter?" needs a query, not a document (AI05 L5).

The cost arithmetic might surprise you

Compare a 150,000-token cached knowledge base with RAG sending 20 chunks of 800 tokens each. Using Opus 5.5's input price of $4 per million tokens and a cache read of 0.05×:

Approach	Input tokens per question	Input cost per question
Cached whole knowledge base (read)	150,000	**$0.030**
RAG, 20 chunks, uncached	16,000	**$0.064**

Don't over-read this

Under these assumptions, reading far more text costs less, because cache reads are so cheap. But the first write costs $0.75, it needs traffic often enough to keep the cache warm, and RAG's own costs (embeddings, a store, a reranker) aren't counted. Very long context can also dilute attention (AI02 L6). So this doesn't prove that stuffing wins. It shows that "RAG is cheaper because it sends less" is an assumption worth checking.

What RAG really is

The original RAG paper (Lewis and colleagues, 2020) described models that "combine pre-trained parametric and non-parametric memory for language generation": a generative model, plus an external searchable index.

Memory	What it is	Updated by
Parametric	Knowledge in the weights	Training
Non-parametric	Documents in an index	Re-indexing

That separation is the whole appeal: **you can change what the system knows without touching the model.**

What RAG doesn't fix

Assumption	Reality
"RAG stops hallucination"	It reduces it. Models can still misread or ignore sources (AI02 L6)
"Retrieval finds the answer"	Chunking and search can both miss it (AI05 L2–L3)
"More chunks are safer"	More cost, and more dilution
"It handles any question"	Weak on aggregations and whole-corpus questions (AI05 L5)
"Documents are safe"	Retrieved text can carry injected instructions (AI09 L2)
"A filter on the answer protects data"	By then, the model has read it (HLD M21B)

What real systems look like

Here's a published production system, and it's more instructive than any idealized diagram. LinkedIn's engineers described their generative AI product:

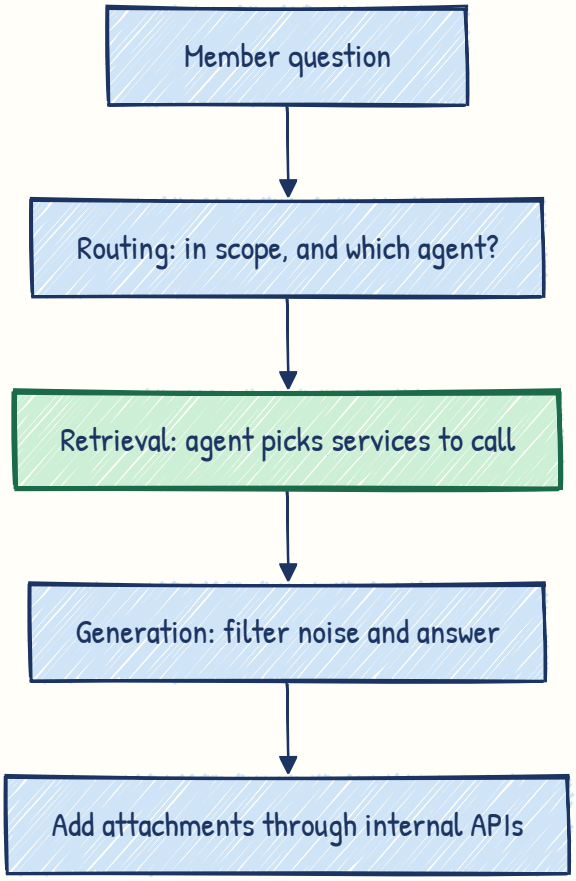


Figure 2. LinkedIn's published three-step pipeline. Small models route, retrieval is tuned for recall by choosing services to call, and larger models generate with an eye on precision.

Their finding	What it teaches
The basic pipeline ran "in just a few days"	A demo isn't the hard part
Reached 80% of the experience in the first month, then spent about four more months trying to pass 95%	Quality's last stretch dominates the schedule
"Underestimated the challenge of detecting and mitigating hallucinations"	Plan for it from day one
Human linguists evaluated up to 500 daily conversations	Evaluation is real work
About 10% of LLM outputs had invalid parameter formats, cut to about 0.01% with a defensive parser and prompt hints	Validate structure (AI03 L3)
Retrieval included calling services such as People Search and Bing	Retrieval isn't always a vector search

Note the last row. In production, "retrieval" often means a mix of search indexes and API calls. It's a broader idea than a vector database.

How Anthropic approaches it

Problem	Anthropic's approach
Teams over-build	The 200K-token guidance, and starting simple (AI06 L1)
Chunks lose context	Contextual Retrieval (AI05 L3)
Claims can't be checked	The Citations feature (AI05 L4)
Repeated context is costly	Prompt caching (AI03 L5)
Agents need more than one search	Just-in-time retrieval through tools (AI03 L6)

Anthropic's context engineering post also describes a hybrid: some content supplied up front, and the rest loaded when needed using tools. Claude Code works this way, with `CLAUDE.md` loaded up front, and files searched for on demand.

The production architecture

This is a recommended reference design, not a specific company's system. AI05 L7 expands it fully.

Baseline: stuff everything, always. It's fine until the corpus grows or permissions differ.

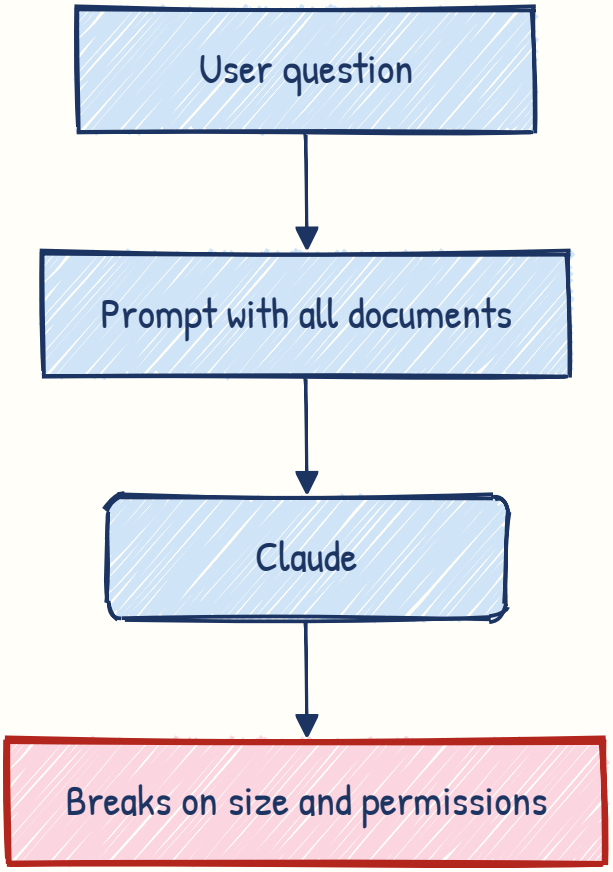


Figure 3. Baseline. The whole corpus goes in the prompt, which works only while it's small and the same for everyone.

Improved: plain RAG. It scales. New problem: **it's now possible to retrieve the wrong thing**, and the model answers confidently anyway.

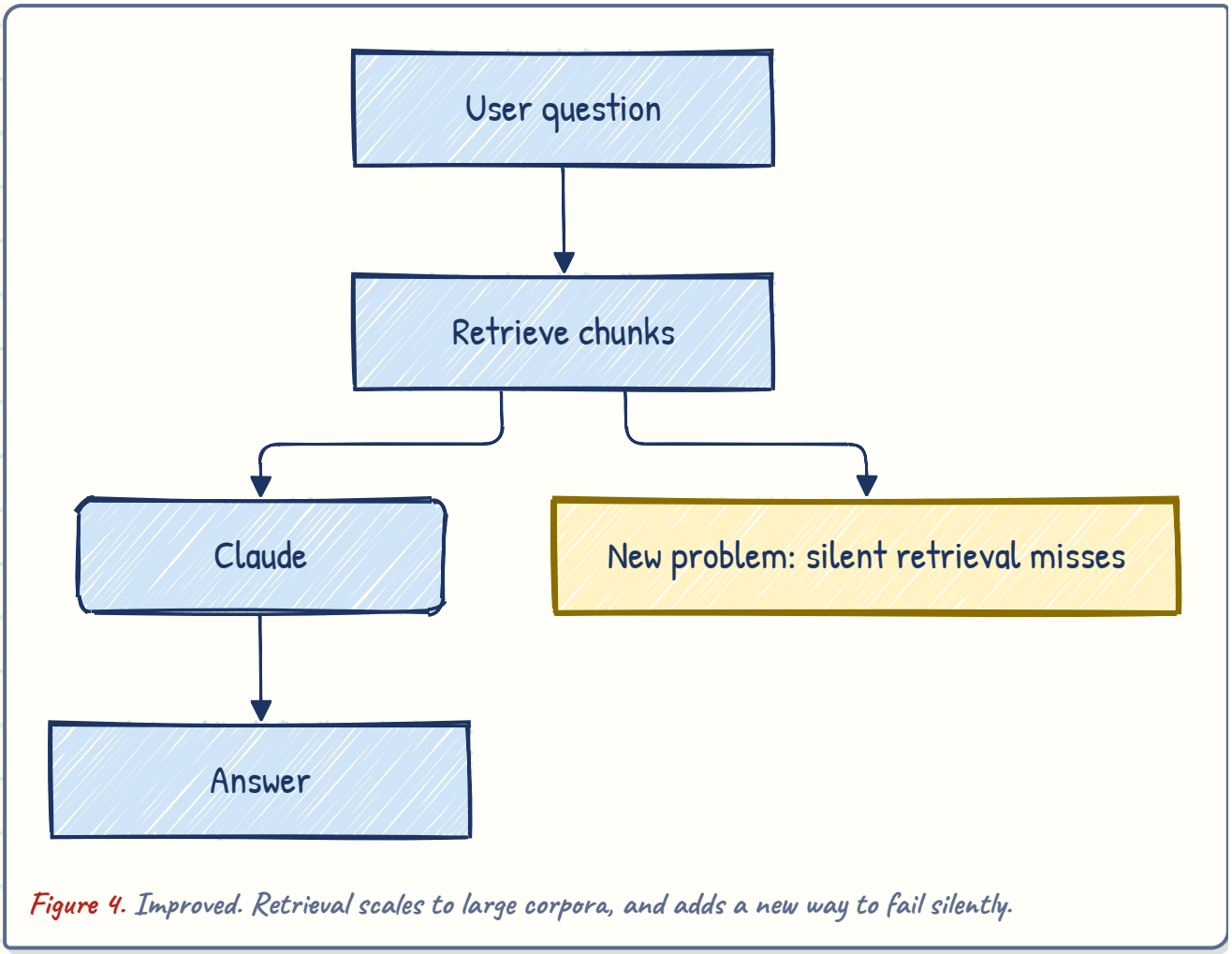


Figure 4. Improved. Retrieval scales to large corpora, and adds a new way to fail silently.

Production: route to the right source, and measure each stage.

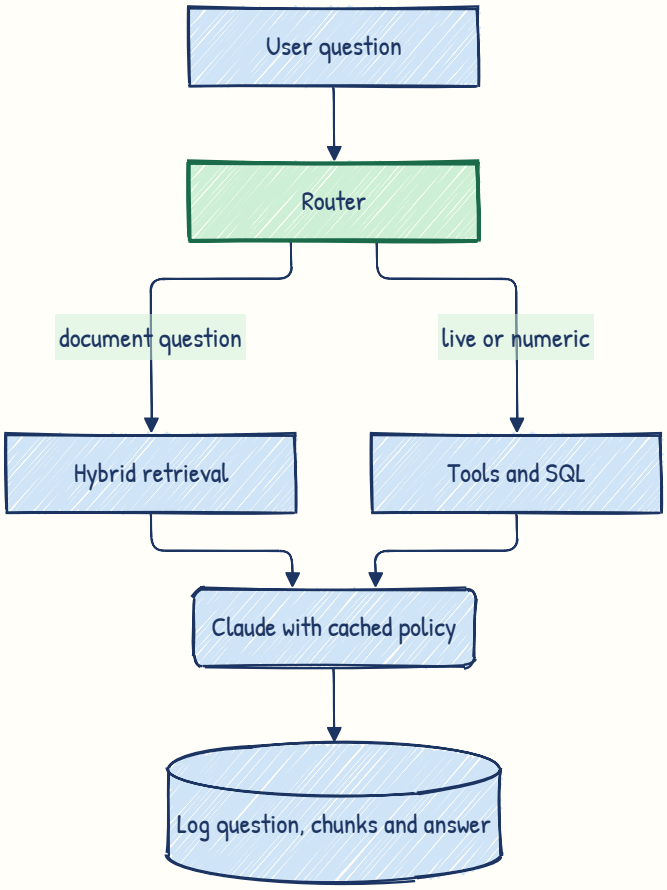


Figure 5. Production. A router sends each question to the right knowledge source, cached common knowledge stays in the prompt, and each stage's output is logged for evaluation.

Component	Why it exists	How it communicates
Router	Different questions need different sources	Rules, or a small model
Hybrid retrieval	Documents too large to stuff	Calls the search service
Tools	Live data and calculations	Tool use (AI06 L3)
Cached policy	Always-relevant knowledge	System prompt with `cache_control`
Logs	Tells you which stage failed (AI05 L6)	Async

Failures

Failure	Containment
Retrieval misses	Log chunk IDs, and allow "I don't know"
Router chooses the wrong source	Evaluate routing separately. Allow a fallback search
Stale index	Show document dates, and monitor sync lag
Cache goes cold	Accept the occasional write, or warm it

Java

A simple router can start as plain code. Rules are testable, cheap and explainable.

```
enum Source { DOCUMENTS, LIVE_DATA, ANALYTICS }

static Source route(String question) {
    String q = question.toLowerCase(Locale.ROOT);
    if (q.matches(".*\\b(how many|total|average|sum of)\\b.*")) {
        return Source.ANALYTICS;
    }
    if (q.matches(".*\\b(my order|my balance|status of)\\b.*")) {
        return Source.LIVE_DATA;
    }
    return Source.DOCUMENTS;
}
```

Real output from `QuestionRouter.java`:

```
ANALYTICS   How many refunds did we issue last quarter?
LIVE_DATA   What's the status of order 7781?
DOCUMENTS   What does the leave policy say about adoption?
ANALYTICS   What's the total notice period for managers?
```

The last line is wrong, and it's a useful failure: "total" sent a policy question to analytics. Rules like these need an eval too, and a fallback when the chosen route finds nothing. **Move to a small model only when evals show that rules aren't enough.** LinkedIn uses small models for this step, at their scale and variety.

Trade-offs

Decision	Gain	Cost
Stuffing with caching	No retrieval failures, simple	Size limit, shared permissions
RAG	Scale, per-user permissions, fresh data	Pipeline, and silent misses
Tools	Live and exact	Each one needs building and securing
Router	Right source per question	Another failure point

When not to use RAG: a small, stable, uniformly permitted corpus; live data; or aggregations.

Common mistakes

1. **Starting with a vector database.** Count your tokens first.
2. **Using RAG for "how many" questions.** That's a query.
3. **Assuming RAG stops hallucination.** It reduces it.
4. **Assuming fewer tokens means cheaper.** Cache reads change the arithmetic.
5. **Treating retrieval as vector search only.** It may be APIs too.
6. **Judging by the demo.** LinkedIn's first 80% took a month, and the rest took far longer.

What to say in an interview

"First I'd count the tokens. Under about 200,000, and with the same access for everyone, I'd try a cached prompt, because it can't fail at retrieval. Live data goes behind tools, and aggregations go to SQL. I'd build RAG for large or permissioned collections, and I'd log retrieved chunks from day one, since LinkedIn's write-up shows the last stretch of quality taking most of the time."

Certification lens

How this lesson maps to CCAR-F

RAG isn't an exam topic by name. Related ideas are tested: **MCP resources and tools for live data** (TS 2.4), **incremental retrieval rather than loading everything** (TS 2.5), and **provenance** (TS 5.6).

These are original practice questions for understanding, not exam simulations.

Q Q1 · Component selection · Medium. Pick an approach for each: (a) "What's the status of my order 7781?" (b) "What did the leave policy say about adoption?", with a 40-page policy. (c) "How many tickets did we close last week?" (d) Search across 3 million internal documents with per-team access.

A (a) Tool. (b) Cached prompt. (c) SQL or an analytics tool. (d) RAG with permission filtering inside the search.

Q Q2 · Calculation · Medium. A 100,000-token cached document on Opus 5.5 ($4 per million, reads at 0.05×, 5-minute writes at 1.25×) serves 1,000 questions within the TTL. What's the input cost for the prefix?

A Write: $0.50. Reads: 999 × 100,000 × $0.20 / 1M = $19.98. Total **$20.48**, compared with $400 uncached.

Q Q3 · Best decision · Hard. Your RAG demo answers 80% of questions well after two weeks. Leadership expects 99% in another two weeks. What do you tell them, using evidence?

A LinkedIn reported reaching 80% in the first month, then about four more months trying to pass 95%. So the last stretch dominates. I'd propose measuring by question type, fixing the largest failure group first, restricting the launch to types that already pass, and being clear about which rate is achievable by when.

Revision - Lesson 1

→ RAG supplies knowledge that won't fit in the prompt, changes, or was never in training.

→ Check first: under about 200,000 tokens, Anthropic suggests a cached prompt instead.

→ Live data goes behind tools, and aggregations go to SQL.

→ Cache reads can make a large cached prefix cheaper than a smaller uncached one.

→ RAG reduces hallucination. It doesn't remove it.

→ A retrieval miss produces a confident wrong answer.

→ LinkedIn: a pipeline in days, 80% in a month, and about four more months towards 95%.

→ Real retrieval often mixes search indexes and API calls.

→ Route by question type, and log chunks for evaluation.

Your notes